Reviewer Pack — Minimal Rank of Tropical Theta Functions vs DPLL Refutation …

Entry 2e2a91b88a9e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 14:45:43 UTC

Minimal Rank of Tropical Theta Functions vs DPLL Refutation Tree Width

Entry ID: `2e2a91b88a9e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 14:45:43 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Theta Functions) **Field B** (complexity object): Complexity Theory (DPLL Refutation Complexity)

Statement:

> {'quantitative_relation': 'For all satisfiable 3-CNF formulas F with n variables, the rank of the tropical theta function associated with F is upper-bounded by O(log n).', 'counterexample_formulation': 'There exists a satisfiable 3-CNF formula F with n variables such that the rank of the tropical theta function associated with F is Ω(n^2).'}

Rationale (proposer's reasoning):

> {'explanation': 'Tropical theta functions capture the arithmetic structure of semigroups and rings, which are closely related to Boolean satisfiability problems. A deeper understanding of their ranks could reveal insights into the complexity of DPLL refutation trees.', 'motivation': 'If the conjecture holds, it would suggest a direct connection between tropical geometry and complexity theory, potentially leading to new algorithms or lower bounds.'}

Taxonomy category: `TROPICAL_FOURIER_ANALYSIS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c3bb7d15cf02d71e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a 3-CNF formula F with n variables, if the rank of its tropical theta function exceeds O(log n), or if there exists a significant positive correlation between the ranks and the DPLL refutation tree widths across all seeds, then the conjecture is falsified. Otherwise, it is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND theta functions AND DPLL refutation tree width - DPLL complexity AND tropical theta function rank lower bound - 3-CNF formulas AND satisfiability AND tropical geometry theta functions rank

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/0712.3205v2] Tropical theta characteristics - [http://arxiv.org/abs/1309.355 Tropical Amplitudes - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2404.02414v2] A simple lower bound for the complexity of estimating partition functions on a quantum computer - [http://arxiv.org/abs/cs/9906008v2] A Lower Bound on the Average-Case Complexity of Shellsort - [http://arxiv.org/abs/2305.12754v3] q-difference equations satisfied by the universal mock theta functions - [http://arxiv.org/abs/2002.10808v1] Generalizing tropical Kontsevich's formula to multiple cross-ratios

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
import sys

def generate_3cnf(n):
    clauses = []
    for _ in range(2 * n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if all(clause[i] == -clause[j] for i in range(n) for j in range(i + 1, n)):
            continue
        clauses.append(clause)
    return clauses

def dpll_refutation_tree_width(formula):
    def dpll(clauses, assignment):
        if not clauses:
            return 0
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
```

```
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            new_clauses = [c for c in clauses if literal not in c and -literal not in c]
            return max(dpll(new_clauses, new_assignment), dpll(new_clauses, {**new_assignment, lit
        pure_literal = next((l for l in range(1, 2 * n + 1) if (all(l in c or -l in c for c in cla
        if pure_literal:
            new_assignment = assignment.copy()
            new_assignment[pure_literal] = True
            new_clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
            return max(dpll(new_clauses, new_assignment), dpll(new_clauses, {**new_assignment, pur
        literals = [l for l in range(1, 2 * n + 1) if any(l in c or -l in c for c in clauses)]
        literal = random.choice(literals)
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        return max(dpll(new_clauses, new_assignment), dpll(new_clauses, {**new_assignment, literal
    return dpll(formula, {})

def tropical_theta_rank(formula):
    n = len(formula[0])
    matrix = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
    for clause in formula:
        max_row = None
        max_val = -math.inf
        for i, row in enumerate(matrix):
            val = sum(row[abs(lit) - 1] * (1 if lit > 0 else -1) for lit in clause)
            if val > max_val:
                max_val = val
                max_row = i
        if max_row is not None:
            factor = matrix[max_row][max_row]
            for j in range(n):
                matrix[j][max_row] /= factor
            for i, row in enumerate(matrix):
                if i != max_row:
                    factor = matrix[i][max_row]
                    for j in range(n):
                        matrix[i][j] -= factor * row[max_row]
    rank = sum(1 for row in matrix if any(val != Fraction(0) for val in row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 20, 40]
    results = []

    for n in n_values:
        formula = generate_3cnf(n)
        theta_rank = tropical_theta_rank(formula)
        refutation_width = dpll_refutation_tree_width(formula)
        results.append({
            "n": n,
            "theta_rank": theta_rank,
            "refutation_width": refutation_width
        })
```

```
    mean_theta_rank = sum(result["theta_rank"] for result in results) / len(results)
    std_theta_rank = math.sqrt(sum((result["theta_rank"] - mean_theta_rank) ** 2 for result in res
    correlation = sum((result["theta_rank"] - mean_theta_rank) * (result["refutation_width"] - sum

    conjecture_holds = correlation < 0
    counterexample = "" if conjecture_holds else "High correlation between theta rank and refutati

    return {
        "metric_name": "Correlation between tropical theta rank and DPLL refutation tree width",
        "metric_value": correlation,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(seed) for seed in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_correlation = sum(result["metric_value"] for result in results) / len(results)
    std_correlation = math.sqrt(sum((result["metric_value"] - mean_correlation) ** 2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_correlation} std={std_correlation} support_fraction={
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_correlation} std={std_correlation} support_fraction={
    else:
        first_failing_seed = next((result["seed"] for result in results if not result["conjecture_
        print(f"RESULT: FALSIFIED counterexample='High correlation between theta rank and refutati
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a70080b.py", line 110, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a70080b.py", line 83, in run_trial
    refutation_width = dpll_refutation_tree_width(formula)
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a70080b.py", line 50, in dpll_refutati
    return dpll(formula, {})
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a70080b.py", line 38, in dpll
```

```
      pure_literal = next((l for l in range(1, 2 * n + 1) if (all(l in c or -
l in c for c in clauses)) and not any(-l in c for c in clauses)), None)
                                                  ^
NameError: name 'n' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means we cannot confirm whether the conjecture is supported or falsified. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11887
2	propose	ollama_remote	glm4:latest	0	0	10591
3	preregistration	ollama_remote	glm4:latest	0	0	5844
4	novelty	ollama_remote	glm4:latest	0	0	4575
5	novelty	ollama_remote	glm4:latest	0	0	6122
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15998
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12640
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12834
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16094
10	judge	ollama_remote	glm4:latest	0	0	8104

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 104687 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2e2a91b88a9e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2e2a91b88a9e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2e2a91b88a9e.tar.gz` (if generated)